TECHNICAL REQUIREMENTS DOCUMENT (TRD)

IRoad Platform - B2B Logistics SaaS

Phase 1 (MVP) Core Architecture & Engineering Standards

DOCUMENT CONTROL & METADATA

Attribute	Details
Document Name	IRoad Technical Requirements Document (TRD)
Project	IRoad Logistics Platform (B2B SaaS)
Version	1.0.0 (Release Candidate - Pending PCS Alignment)
Date	March 2026
Target Audience	Development Vendor, Solutions Architects, QA Engineers
Confidentiality	**Strictly Confidential.** Not to be distributed or duplicated without explicit authorization from the Platform Owner.

APPROVED TECHNOLOGY STACK

- **Database:** PostgreSQL (Multi-Schema Topology)
- **Caching & Sessions:** Redis (Persistence Enabled)
- **Backend API:** NestJS (Node.js, TypeScript)
- **Web Frontend:** React.js (TypeScript, SPA)
- **Mobile App:** React Native (iOS & Android, Offline-First SQLite)
- **Infrastructure:** Docker & Docker Compose (Containerized delivery)
- **Storage:** S3-Compatible Object Storage

0. Drafting & Formatting Standards

Document Purpose: This document defines the strict drafting guidelines for the IRoad TRD. It ensures the final TRD acts as an unambiguous, testable, and legally binding engineering contract between the Platform Owner and the Development Vendor.

1. **Normative Terminology (RFC 2119):** All requirements must use standard engineering terminology (`MUST`, `SHALL`, `MUST NOT`, `SHOULD`).
2. **Testability & Measurability:** Vague statements are strictly prohibited. Every requirement must be verifiable during User Acceptance Testing (UAT).
3. **Tech-Stack Specificity:** The TRD assumes the approved stack: PostgreSQL, NestJS, React, React Native, and Docker.
4. **Defensive Architecture & Immutability:** Always define technical boundaries. Clearly state when to use Soft Deletes vs. Hard Deletes, and enforce Snapshot Immutability.

5. **Strict Separation of Concerns:** Complex features must be broken down by UI/Frontend Duty, Backend/API Duty, and Database Duty.
6. **Consistent Terminology:** Entities must be referred to by a single name (e.g., `Super Admin`, `Tenant`, `Booking`, `Shipment`).

Volume I: Infrastructure, Hosting & Databases

Part 1: Deployment Model, Architecture & Docker

1.1 High-Level Architecture Strategy

The IRoad platform operates as a centralized B2B SaaS platform governing isolated Transporter environments. The system architecture MUST decouple the central governance layer (Super Admin) from the execution layer (Tenant/Transporter).

1.1.1 Architectural Flow Visualization

```
graph TD
    subgraph Client Layer
        A[Super Admin Web - React]
        B[Tenant Web - React]
        C[Driver App - React Native]
    end

    subgraph API Gateway & Routing
        D[Reverse Proxy / Load Balancer]
    end

    subgraph Application Layer - NestJS
        E[Super Admin Services]
        F[Tenant Operational Services]
        G[Shared Services/API Bridges]
    end

    subgraph Persistence Layer
        H[(Redis Cache & Sessions)]
        I[(PostgreSQL Master DB)]
        J[("PostgreSQL Tenant DBs (Isolated Schemas)")]
    end

    A --> D
    B --> D
    C --> D

    D --> E
    D --> F
    D --> G

    E --> I
```

```
    E --> H
    F --> J
    F --> H
    G --> I
    G --> J
```

1.2 Multi-Tenancy & Isolation Model

- The system MUST utilize a **Multi-Schema** architecture within PostgreSQL.
- The system MUST maintain one `master` schema exclusively for Super Admin configurations and Global Master Data.
- Every active Transporter MUST be provisioned an entirely isolated database schema (e.g., `tenant_xyz`).
- **Cross-Schema Query Restriction:** The backend NestJS modules serving Tenant operations MUST NOT query data from another Tenant's schema under any circumstances.

  🛑 **CRITICAL RULE: Referential Integrity in Isolation**

  A Tenant's local operational record MUST NOT contain a direct Foreign Key pointing to a record in the `master` schema if that master record is subject to dynamic change. Global data MUST be seeded as read-only copies into the Tenant's isolated schema.

1.3 Containerization & Docker Mandate

The vendor MUST deliver all system components as containerized applications. "It works on my machine" is NOT an acceptable delivery state.

1. **Frontend Container:** React applications MUST be containerized using an Nginx-based Alpine Linux image.
2. **Backend Container:** NestJS MUST be containerized using a Node.js Alpine image.
3. **Database Containers:** PostgreSQL and Redis MUST be provisioned using official Docker images.

The vendor MUST provide comprehensive `docker-compose.yml` files (`dev` and `prod`).

1.4 Environment Variables & Configuration Management

The system MUST NOT contain any hardcoded infrastructure configurations, API keys, or database credentials within the source code. All mapping MUST rely strictly on `.env` files.

Part 2: Automated Tenant Provisioning Engine

2.1 Engine Overview & Trigger Mechanism

The Automated Tenant Provisioning Engine transforms a new subscription into a fully functional, isolated, and seeded operational environment for the Transporter without manual database administration.

- **Trigger:** The engine MUST be triggered by a REST API call or an automated webhook.
- **Idempotency:** The API MUST be idempotent. It MUST NOT overwrite or recreate an existing schema.

2.2 The Provisioning Workflow

The NestJS backend MUST execute the following sequence:

```
graph TD
    A[Trigger: Provision API Called] --> B{Begin DB Transaction}
    B --> C[1. Create Tenant Master Record in 'master']
    C --> D[2. Execute CREATE SCHEMA 'tenant_xyz']
    D --> E[3. Run DB Migrations for 'tenant_xyz']
    E --> F[4. Seed Global Master Data]
    F --> G[5. Create Default Tenant Admin User]
    G --> H[6. Dispatch Welcome Email via Comm Engine]
    H --> I{Commit DB Transaction}

    I -->|Success| J((Tenant Active))
    I -->|Failure| K[ROLLBACK ALL Changes]

    style K fill:#ffebee,stroke:#d32f2f,color:#c62828
    style J fill:#e8f5e9,stroke:#2e7d32,color:#2e7d32
```

> 🛑 **CRITICAL RULE: Atomic Rollback**
>
> The entire steps 1 through 5 MUST be executed within a unified database transaction block. If schema creation or seeding fails, the system MUST `ROLLBACK` the creation of the Tenant Master Record.

2.3 Data Seeding Strategy (Seed & Sync)

The system MUST seed the following Super Master Data into the new schema: `Countries Master`, `Currencies Master`, `Tax Codes`. The seeder script MUST NOT alter the primary keys (UUIDs/Codes) during the copy process.

2.4 Default Account & Handover

The final step handles access granting. The system MUST create one `UserAccount` record inside the newly created Tenant schema assigned the immutable `Super Admin` role for that specific TMS. The backend MUST dispatch a secure, time-limited Activation Token (JWT) via email to set the password.

2.5 API Contract for Provisioning

The vendor MUST implement the following RESTful API endpoint to trigger the engine.

- **Endpoint:** `POST /api/v1/super-admin/tenants/provision`
- **Authorization:** `Bearer <SuperAdmin_JWT>`
- **Content-Type:** `application/json`

```
{
  "company_name_en": "Fast Transport LLC",
  "company_name_ar": "شركة النقل السريع",
```

```
  "registration_number": "1010123456",
  "primary_email": "admin@fasttransport.com",
  "country_code": "SA",
  "subscription_plan_id": "123e4567-e89b-12d3-a456-426614174000"
}
```

- **Response Expectations:**
- `201 Created`: Schema provisioned, migrations run, seeded, and email dispatched.
- `409 Conflict`: Tenant already exists.
- `500 Internal Server Error`: Provisioning failed, changes rolled back.

Part 3: Database Architecture & Data Sync

3.1 PostgreSQL Multi-Schema Topology

The vendor MUST implement a Multi-Schema topology within a single PostgreSQL database instance. The backend MUST NOT execute `JOIN` queries across different tenant schemas or between a tenant schema and the `master` schema.

3.2 The "Seed & Sync" Protocol (Master to Tenant)

The following entities MUST be replicated as **Read-Only** tables inside every tenant schema: `CountryMaster`, `CurrencyMaster`, `TaxConfig`.

- The NestJS backend MUST implement an Internal Event Bus (Redis Pub/Sub or EventEmitter). When Master data changes, an event broadcasts to update all tenant schemas.
- Tenants MUST NOT have the API permission to modify replicated global data tables within their own schemas.

3.3 Event-Driven Asymmetric Sync (Tenant to Master)

When a Transporter updates their company profile (`OrganizationProfile`), the tenant module MUST publish a `Tenant.ProfileUpdated` event. The Super Admin module MUST listen and asynchronously update the corresponding `SubscriberProfile` record in the `master` schema.

3.4 API Bridging (Shared Modules)

For central modules (e.g., Support Tickets, Billing):

- The Tenant UI MUST communicate directly with the `master` schema via secure **API Bridges**.
- The NestJS backend MUST extract the `tenant_id` from the JWT token and append it as a strict `WHERE tenant_id = ?` clause to the DB query.

3.5 Super Admin Global Analytics (Aggregated Cache)

The Super Admin dashboard requires global KPIs (e.g., Total System Revenue, Total Active Shipments across all tenants).

- **Prohibited Practice:** The NestJS backend MUST NOT execute real-time loops iterating through 100+ tenant schemas to aggregate these numbers dynamically per API request. This will cause catastrophic memory overload (OOM) and database timeouts.
- **Mandatory Architecture:** The vendor MUST implement an **Aggregated Materialized View** or a **Global KPI Cache Table** within the `master` schema.
- **Execution:** A scheduled background job (e.g., running every 1 hour via BullMQ) MUST query the individual tenant schemas, aggregate the KPI metrics, and `UPSERT` the results into the `master.GlobalAnalyticsCache` table. The Super Admin React frontend MUST query this cache table exclusively for cross-tenant reporting.

Part 4: Storage & Attachment Management

4.1 Global Storage Architecture

- **Prohibited Practice:** The NestJS backend MUST NOT store binary files (BLOBs) directly within PostgreSQL.
- **Mandatory Architecture:** The vendor MUST utilize an S3-compatible Object Storage service. PostgreSQL MUST only store metadata and the relative URI.

4.2 Tenant Isolation & Security (Pre-Signed URLs)

- Files MUST NOT be publicly accessible via static URLs.
- The NestJS backend MUST generate **Short-lived Pre-Signed URLs** for file viewing and downloading.
- The system SHOULD implement a Direct-to-Storage upload pattern, where the frontend uploads directly to S3 using a Pre-Signed upload URL.

4.3 Storage Quota Enforcement

The NestJS backend MUST track the total byte size of all files associated with a `tenant_id` and evaluate it against `OrganizationProfile.active_max_storage_gb`. If the limit is exceeded, return a `403 Forbidden`.

> 🛑 **CRITICAL RULE: Deletion Protocol**
>
> If an attachment record is hard-deleted from PostgreSQL, the NestJS backend MUST simultaneously issue a `DELETE` command to the S3 Object Storage to physically remove the file.

Part 5: Backups, Disaster Recovery & Archiving

5.1 Backup Strategy & Subscription Tiers

The vendor MUST implement automated backups aligning with commercial tiers:

- **Standard:** Daily Automated Schema Dumps (24h RPO).

- **Extended:** Daily Dumps + 6-Hour Incremental Backups (6h RPO).
- **Premium:** Continuous WAL Archiving (Point-in-Time Recovery).
- The vendor MUST provide scripts to restore a *single* `tenant_id` schema from backup without affecting others.

5.2 Automated Archiving Policy (Cold Storage)

- `Communication Logs` MUST be retained for 90 Days.
- A NestJS Cron Job MUST query logs older than 90 days, serialize them (JSON Lines/CSV + GZIP), upload to Cold Object Storage (e.g., S3 Glacier), verify the `200 OK` response, and ONLY THEN execute `DELETE` in PostgreSQL.

5.3 The 12-Month Master Audit Log Rule

The `MasterSystemAuditLog` table stores heavy `JSONB` payloads. Without pruning, it will bloat the database and degrade performance.

- **Retention Period:** 12 Months.
- **Execution:** The Archiving Cron Job MUST query audit logs older than 365 days, serialize them to a compressed format (e.g., JSON Lines + GZIP), and upload them to Cold Object Storage (e.g., AWS S3 Glacier).
- The system MUST NOT execute the `DELETE` command in PostgreSQL until a `200 OK` response confirms the successful upload to the cold storage.

End of Volume I.

Volume II: Security, Audit & Integrity

Part 6: Security Policies, Authentication & Sessions

6.1 Authentication Architecture (JWT & Redis)

The IRoad platform serves three distinct user domains: `Super Admin` (Platform Owner), `Tenant Web User` (Transporter Back-office), and `Driver` (Mobile App). The vendor MUST implement a robust, stateless-yet-revocable authentication architecture.

6.1.1 Token Strategy

- The NestJS backend MUST utilize **JSON Web Tokens (JWT)** for authentication across all domains.
- The JWT payload MUST minimally contain: `user_id`, `role_id`, `tenant_id` (if applicable), and `domain_type` (Admin, Tenant, Driver).
- The backend MUST NOT store sensitive data (like passwords or permissions matrices) inside the JWT payload.

6.1.2 The Caching Layer (Session Registry)

While JWT is inherently stateless, the system requires active session monitoring and instant revocation capabilities.

- The vendor MUST utilize **Redis** as the active session registry.
- Upon successful login, the NestJS backend MUST store the generated JWT's unique identifier (`jti` or session ID) in Redis, mapped to the user's ID and `tenant_id`.
- Every authenticated API request MUST pass through a NestJS `AuthGuard` that validates the JWT signature AND checks Redis to ensure the session is still marked as active.
- **Redis Persistence Mandate:** To prevent the catastrophic loss of all active user and driver sessions in the event of a server restart or container crash, the vendor MUST configure the Redis instance with data persistence enabled (utilizing RDB snapshots and/or AOF - Append Only File). Operating Redis in a volatile, in-memory-only mode for the session registry is STRICTLY PROHIBITED.

6.2 Configurable Time-To-Live (TTL) & Lockouts

Hardcoded session limits are strictly prohibited. The system MUST enforce dynamic security configurations defined by the Super Admin in the Control Panel (`FRM-CP-11-01` and `FRM-CP-11-02`).

6.2.1 Session Expiry Rules

The Redis TTL for sessions MUST dynamically adhere to the following configurations:

- **Admin Session TTL:** Default `240 minutes` (4 Hours).
- **Tenant Web Session TTL:** Default `12 hours`.
- **Driver App Session TTL:** Default `30 days` (To accommodate offline/remote travel scenarios).

6.2.2 Brute-Force Protection

The NestJS authentication controller MUST implement rate-limiting and brute-force protection logic.

- If a user exceeds the `max_failed_logins` limit (Default: 3 for Admins, 5 for Tenants), the account MUST be temporarily locked.
- The lockout duration MUST be enforced based on the `lockout_duration_minutes` configuration.

6.3 The "Kill Switch" (Absolute Override Protocol)

The "Kill Switch" is a mission-critical commercial and security tool. It guarantees IRoad's absolute sovereignty over platform access.

6.3.1 Trigger & Execution

When a Super Admin changes a Tenant's status to `Suspended` (due to non-payment or severe violation), the system MUST instantaneously terminate all access for that Tenant's entire workforce.

```
sequenceDiagram
    autonumber
    actor SuperAdmin as Super Admin
    participant CP as Control Panel API (NestJS)
    participant Redis as Redis Cache
    participant AuthGuard as Tenant API AuthGuard
    participant Users as Tenant Users & Drivers

    SuperAdmin->>CP: 1. Update Tenant Status to 'Suspended'
    CP->>CP: 2. Commit status change to PostgreSQL Master DB
    CP->>Redis: 3. EXECUTE: Mass Revoke (Delete all keys matching
`tenant_id:XYZ:*`)
    Redis-->>CP: 4. Keys Deleted
    Users->>AuthGuard: 5. Subsequent API Request (with previously valid JWT)
    AuthGuard->>Redis: 6. Check Token Validity
    Redis-->>AuthGuard: 7. Token Not Found (Revoked)
    AuthGuard-->>Users: 8. HTTP 401 Unauthorized (Force Logout UX)
```

> 🛑 **CRITICAL RULE: Instantaneous Revocation**
>
> The Kill Switch MUST NOT rely on waiting for the JWTs to naturally expire. The revocation MUST be executed at the caching layer (Redis) and MUST take effect on the very next API request made by any user or driver associated with the suspended `tenant_id`.

6.4 Authentication & Access Log

To detect malicious activities and support security audits, the backend MUST maintain an immutable access log (`FRM-CP-11-04`).

- **Execution:** Every login attempt (Success or Failure) and logout event MUST be recorded asynchronously.

- **Storage:** This log resides in the `master` schema.

|

| **Field Key** | **Data Type** | **Req.** | **Description / Validation Logic** |

| `log_id` | `UUID` | **Yes** | Auto-generated PK. |

| `attempt_type` | `Enum` | **Yes** | `[Login, Logout, Token_Refresh]` |

| `status` | `Enum` | **Yes** | `[Success, Failed, Blocked]` |

| `user_domain` | `Enum` | **Yes** | `[Admin, Tenant_User, Driver]` |

| `email_used` | `Text` | **Yes** | Captured even if the user does not exist in the DB. |

| `ip_address` | `Text` | **Yes** | Source IP of the attempt. |

| `user_agent` | `Text` | **Yes** | Browser or Mobile App metadata. |

| `timestamp` | `DateTime` | **Yes** | Exact system time of the attempt. |

6.5 Multi-Device Policy (Concurrent Sessions)

The system MUST support concurrent sessions (e.g., a Driver logging in from a new phone, or an Ops User logging in from a desktop and a tablet simultaneously).

- The Redis registry MUST support multiple active `jti` tokens per `user_id`.
- The Super Admin "Active Sessions Monitor" (`FRM-CP-11-03`) MUST display all individual active sessions, allowing the Admin to invoke a targeted revocation (killing a specific device) without affecting the user's other active devices.

Part 7: Transactional Immutability & Master Audit Log

7.1 Operational Snapshot Policy (OSP v1.0)

Financial and operational records MUST NOT change retroactively if the underlying master data (like a Client's address or a Tax Rate) is updated. The vendor MUST implement the "Snapshotting" pattern.

7.1.1 The Snapshot Trigger

- When a `SalesOrder` transitions to `Confirmed`, or a `StandardInvoice` is `Issued`, the NestJS backend MUST execute a snapshot routine.
- The backend MUST copy the literal values (e.g., `Client_Name`, `Tax_Percentage`, `Currency_FX_Rate`, `Driver_Name`) directly into the transaction table columns (e.g., `snapshot_client_name`, `applied_tax_rate`).
- Utilizing relational queries (`JOIN ClientMaster`) to calculate the grand total of an older invoice on the fly is STRICTLY PROHIBITED.

7.1.2 UI/UX Enforcement

- Once a record is snapshotted, the NestJS backend MUST reject any `PATCH` or `PUT` requests attempting to modify these commercial fields.
- The React frontend MUST render these fields as strictly `Read-only` disabled inputs for issued documents.

7.2 Currency & FX Snapshot Enforcement

IRoad utilizes a Dual-Currency system (Transaction Currency vs. Base Currency).

- The backend MUST calculate and store `base_amount` for every financial line item at the time of creation.
- **Formula:** `base_amount` = `transaction_amount` * `fx_rate_snapshot`.
- **Immutability:** If a Super Admin updates the exchange rate in `CurrencyMaster` tomorrow, the historical `SalesOrder` or `StandardInvoice` MUST NOT recalculate its `base_amount`.

7.3 Master System Audit Log (The "Black Box")

Every administrative change to master data, subscription plans, and security configurations MUST be recorded in an immutable ledger in the `master` schema (`FRM-CP-12-01`).

- **Execution:** The logging MUST happen via backend Interceptors or Lifecycle Hooks (e.g., TypeORM `@AfterUpdate`).
- **Payload Storage:** The log MUST contain `old_payload` and `new_payload` stored strictly in `JSONB` format to allow dynamic querying regardless of the entity altered.

> 🛑 **CRITICAL RULE: DB Engine Privilege Restrictions** To prevent "Inside Jobs" or malicious alterations of the audit log, the PostgreSQL user account utilized by the NestJS application MUST ONLY be granted `INSERT` and `SELECT` privileges on the `MasterSystemAuditLog` table. `UPDATE`, `DELETE`, and `TRUNCATE` commands for this specific table MUST be explicitly revoked at the database engine level.

7.4 Archiving & Cold Storage

To prevent database bloat, the `MasterSystemAuditLog` is subject to a strict 12-month retention policy. Older records MUST be automatically compressed and moved to Cold Object Storage as defined in **Part 5.3.3**.

Part 8: Global Security Standards & Compliance

8.1 OWASP Top 10 Mitigation Mandate

The vendor MUST engineer the application defensively against common vulnerabilities.

| **Vulnerability** | **Frontend (React) Mitigation** | **Backend (NestJS/Postgres) Mitigation** |

| **Injection (SQLi)** | N/A | The backend MUST NOT use raw SQL string concatenation. All database queries MUST use parameterized queries via the ORM. |

| **Cross-Site Scripting (XSS)** | React's native JSX escaping MUST be utilized. Use of `dangerouslySetInnerHTML` is STRICTLY PROHIBITED. | NestJS MUST use `class-validator` to strip HTML tags and sanitize incoming JSON payloads. |

| **Security Misconfiguration** | N/A | NestJS MUST implement the `helmet` middleware to set secure HTTP headers. |

| **DDoS / Brute Force** | N/A | The `AuthGuard` MUST implement `@nestjs/throttler` to enforce strict rate limiting on public endpoints. |

8.2 Data Encryption (At Rest & In Transit)

- **Encryption in Transit:** All API communication MUST be encrypted using TLS 1.2 or higher (HTTPS). The backend MUST reject non-secure HTTP requests.
- **Encryption At Rest (Sensitive Data):** The `credentials_payload` containing API keys for Third-Party Payment Gateways MUST NOT be stored in plain text. The NestJS backend MUST implement a dedicated Crypto Service utilizing `AES-256-GCM` to encrypt the payload before DB insertion and decrypt it upon retrieval in memory.

8.3 Regulatory Compliance (ZATCA Readiness)

To ensure the platform can operate legally in the GCC, specifically Saudi Arabia, the core engine MUST adhere to basic e-invoicing constraints.

- **TIN Snapshotting:** Tax Identification Numbers (TIN) of both the Transporter and the Client MUST be permanently snapshotted in the invoice header at the time of issuance.
- **Line-Item Tax Calculation:** Taxes MUST be calculated per line item (quantity * unit_price), rounded to 2 decimal places, and then aggregated to the header. Calculating tax only at the grand total level is PROHIBITED.
- **Arabic Native Support (UTF-8):** The PostgreSQL database and the NestJS PDF generation service MUST natively support `UTF-8` encoding. All generated invoice PDFs MUST perfectly render Arabic characters right-to-left (RTL) without disconnected letters.

End of Volume II.

Volume III: Logic, Integration & Applications

Part 9: Integrations & External Gateways

9.1 Dynamic Gateway Architecture

The IRoad platform MUST interact with multiple third-party providers for Payments (e.g., Stripe, Moyasar) and Communications (e.g., SendGrid, Unifonic).

To ensure the platform can scale without requiring backend code deployments for every new provider, the vendor MUST implement an **Adapter Pattern** combined with a dynamic database-driven configuration. Hardcoding API keys or provider-specific logic in the main business services is STRICTLY PROHIBITED.

9.2 Payment Gateway Integration

The system MUST process subscription payments and wallet top-ups dynamically based on the configurations defined in `PaymentGatewaysAPI` and `PaymentMethodsConfiguration`.

9.2.1 Dynamic Credentials Payload

- The NestJS backend MUST retrieve the provider's API keys from the PostgreSQL database.
- The `credentials_payload` (`JSONB`) MUST be decrypted at runtime using the internal Crypto Service before being passed to the payment adapter.
- **Payload Structure:** The JSON structure MUST be flexible enough to support varying gateway requirements (e.g., `{ "public_key": "...", "secret_key": "...", "profile_id": "..." }`).

9.2.2 The Checkout Router Logic

When a Tenant initiates a checkout from the React frontend, the NestJS backend MUST evaluate the active payment methods.

- **Filter Rule:** The backend MUST return only the methods where `is_active = True` AND the Order's transaction currency exists within the method's `supported_currencies` JSON array.
- **Online Gateway Flow:** If `method_type = Online_Gateway`, the backend MUST initialize a secure payment session with the respective third-party provider and return the session ID/URL to the React frontend.
- **Offline Bank Flow:** If `method_type = Offline_Bank`, the backend MUST bypass third-party APIs, display the configured `IRoadBankAccounts`, and set the Order status to `Pending Payment` awaiting manual Super Admin approval.

9.2.3 Webhook Handling & Idempotency

Payment gateways communicate asynchronously via Webhooks to confirm payment success.

🛑 **CRITICAL RULE: Webhook Security & Idempotency**

1. **Signature Validation:** The NestJS webhook controller MUST implement a specific Guard to validate the cryptographic signature of the incoming webhook payload using the provider's webhook secret. Unsigned or invalid requests MUST be rejected with `401 Unauthorized`.
2. **Idempotency:** Webhooks can be delivered multiple times. The backend MUST check if the associated `SubscriptionOrder` or `FinancialTransaction` has already been marked as `Completed` or `Paid`. If yes, the system MUST return `200 OK` without duplicating the financial ledger entries.

9.3 Communication Engine (SMS & SMTP)

The Centralized Communication Engine dispatches OTPs, invoices, and system alerts.

9.3.1 System Events Mapping

- **Immutability of Triggers:** The list of system events (e.g., `OTP_Requested`, `Invoice_Paid`) MUST be hardcoded in the NestJS backend as an `Enum` or constant list. The Super Admin MUST NOT be able to create new system triggers dynamically.
- **Shortcode Injection:** The communication service MUST dynamically parse and replace template variables (e.g., `{{otp_code}}`, `{{client_name}}`) with live data before passing the payload to the gateway adapter.

9.3.2 The Fallback Protocol (High Availability)

Delivery failures (e.g., SMS Gateway timeout) MUST NOT block critical business flows like user login or payment confirmations.

```
sequenceDiagram
    autonumber
    participant App as NestJS Core Logic
    participant Router as Events Mapping Engine
    participant SMS as Primary (SMS Gateway)
    participant Email as Fallback (SMTP Gateway)
    participant Log as CommunicationLogs (DB)

    App->>Router: 1. Trigger Event (e.g., `OTP_Requested`)
    Router->>SMS: 2. Attempt Delivery via `primary_channel`
    SMS-->>Router: 3. API Error (503 Service Unavailable / Timeout)
    Router->>Email: 4. Catch Exception -> Execute `fallback_channel`
    Email-->>Router: 5. 200 OK (Email Sent)
    Router->>Log: 6. INSERT Log (Primary: Failed, Fallback: Success)
    Router-->>App: 7. Return Success to Caller
```

📋 **Testability & Definition of Done:**

During UAT, the testing engineer will intentionally alter the SMS gateway API key to an invalid value. When a user requests an OTP, the system MUST seamlessly detect the SMS

failure, immediately dispatch the OTP via the configured Fallback Email template, and record both the failure and the success accurately in the `CommunicationLogs` table.

9.4 Push Notifications Manager

- **Manual Broadcast:** If `trigger_mode = Manual_Broadcast`, the NestJS backend MUST offload the dispatch process to a background job queue (e.g., BullMQ backed by Redis) to prevent blocking the main thread while sending thousands of FCM tokens.
- **System Event Broadcast:** If `trigger_mode = System_Event`, the push notification fires synchronously as part of the event listener's execution.

Part 10: Background Jobs, Cron & State Machines

10.1 Automation Architecture

The IRoad platform requires robust automation to handle time-sensitive state changes and heavy asynchronous tasks without blocking the main Node.js event loop.

The vendor MUST implement two distinct automation mechanisms within the NestJS backend:

1. **Scheduled Tasks (Cron Jobs):** Utilizing `@nestjs/schedule` for time-based daily batch processing.
2. **Message Queues:** Utilizing `BullMQ` (backed by Redis) for event-driven, heavy, or retryable asynchronous tasks.

> 🛑 **CRITICAL RULE: Idempotency & Distributed Locking**
>
> As the backend may run in a multi-instance Docker environment (horizontal scaling), the vendor MUST implement Distributed Locking (e.g., using `redlock` with Redis) for all Cron Jobs to ensure that a scheduled task is NOT executed twice simultaneously by different server instances.

10.2 Scheduled Tasks (Cron Jobs)

The following engines MUST run on daily schedules, strictly respecting the `System Timezone`.

10.2.1 Contract & Compliance Engine

- **Execution Time:** Daily at 00:01 (System Timezone).
- **Derived Status Updates:** The job MUST recalculate and update the derived `status` of all contracts (`Upcoming`, `Active`, `Expired`) based on `start_date` and `end_date`.
- **Action Enforcement:** If a contract is `Expired`, the job MUST read the `expired_contract_handling_mode` setting.
- If `Auto-Deactivate`: Change status to `Inactive`.
- If `Deactivate After Grace`: Change status to `Inactive` if `Today > Grace Expiry`.
- If `Do Nothing`: Only the contract status changes; the entity remains active.

10.2.2 Subscription Billing & Grace Period Engine

- If a Tenant's `subscription_expiry_date` is reached, the job MUST check the global `grace_period_days`.

- If `Today > subscription_expiry_date + grace_period_days`, the job MUST update the `SubscriberProfile.account_status` to `Suspended_Billing`.
- This status change MUST trigger the "Kill Switch" (Part 6) to immediately revoke all Redis JWT sessions for that Transporter.

10.3 Automated State Machines

Status transitions MUST NOT be open to arbitrary manual overrides if a strict workflow exists.

10.3.1 Support Ticket "Ping-Pong" Logic

The backend MUST enforce the following transition triggers automatically upon the creation of a `SupportTicketReply`:

```
stateDiagram-v2
    [*] --> New : 1. Tenant Creates Ticket
    New --> In_Progress : 2. Admin Assigns Ticket
    In_Progress --> Pending_Client : 3. Admin adds Reply (is_internal =
False)
    Pending_Client --> In_Progress : 4. Tenant adds Reply
    In_Progress --> Closed : 5. Admin forces Close
    Pending_Client --> Closed : 5. Admin forces Close
    Closed --> [*]
```

- **Condition 1:** If an Admin adds a reply where `is_internal = True`, the system MUST NOT transition the status to `Pending_Client`. It MUST remain `In_Progress`.
- **Condition 2:** The backend MUST reject any API request from a Tenant attempting to add a reply if the Ticket `status == Closed`.

10.3.2 Operational Status Derivation (Orders)

The `SalesOrder` and `PurchaseOrder` modules MUST derive their `operational_status` automatically based on downstream `Booking` and `Shipment` aggregations.

|

| **Derived Status** | **Business Logic / Aggregation Rules** |

| `Not Started` | `Total Booked QTY == 0` AND `Total Executed QTY == 0` |

| `Partially Booked` | `0 < Total Booked QTY < Total QTY` |

| `Fully Booked` | `Total Booked QTY >= Total QTY` AND `Total Executed QTY < Total QTY` |

| `Partially Executed` | `0 < Total Executed QTY < Total QTY` |

| `Fully Executed` | `Total Executed QTY == Total QTY` |

🛑 **CRITICAL RULE: Cancellation Exclusion** The calculation logic MUST strictly exclude any `Booking` or `Shipment` where `status == Cancelled`. Cancelled movements MUST NOT contribute to the `Booked QTY` or `Executed QTY`.

10.4 Event-Driven Queues (BullMQ)

Synchronous execution of heavy tasks blocks the API and leads to timeouts. The vendor MUST offload the following specific tasks to Redis-backed queues (BullMQ).

1. **Standard Invoice Generation:** PDF rendering MUST be pushed to a worker queue.
2. **Push Notification Broadcasts:** Looping over thousands of Driver FCM tokens MUST be processed by a dedicated BullMQ worker with failure retry policies.
3. **Internal Event Bus Synchronization:** Copying updated profiles from a tenant schema to the `master` CRM MUST be queued to ensure no data is lost during database locks.

Part 11: Driver App Tech Specs & Offline-Safe Mode

11.1 Architectural Philosophy (Execution-Only)

The IRoad Driver Mobile App is NOT a management tool; it is strictly an **Execution Interface**.

- The vendor MUST build the application using **React Native**.
- **Navigation:** The app MUST implement a fixed bottom navigation bar with exactly 4 tabs: `My Jobs`, `History`, `My Wallet`, and `Profile`.

11.2 Offline-First Architecture & Sync Engine

Transport drivers frequently operate in remote areas with poor network coverage. The React Native application MUST NOT crash or block operational execution when the device loses internet connectivity.

11.2.1 Local Storage Requirement

- The app MUST utilize a local database (e.g., **SQLite** or **Realm**) to persist state.
- Upon login or manual refresh (pull-to-refresh), the app MUST pre-fetch and store all "Active" shipments, routing addresses, and `OperationActionConfig` payloads locally.

11.2.2 The Sync Reconciliation Queue

When the driver executes an action (e.g., captures a Digital POD) while offline, the app MUST place the payload into a local "Pending Sync Queue".

```
sequenceDiagram
    autonumber
    participant Driver as Driver (React Native)
    participant LocalDB as SQLite (Local Queue)
    participant Net as Network Listener
    participant API as NestJS Backend

    Driver->>LocalDB: 1. Click "Delivered" (No Internet)
    LocalDB->>LocalDB: 2. Store Action Payload in Queue & Update UI
```

```
    Driver-->>Driver: 3. App shows "Offline Mode - Saved Locally"
    Net->>LocalDB: 4. Event: Internet Connection Restored
    LocalDB->>API: 5. Background Sync: POST Pending Payloads
    API-->>LocalDB: 6. 201 Created (Success)
    LocalDB->>LocalDB: 7. Mark as Synced / Clear Queue
```

11.2.3 Security Override (`require_internet_for_actions`)

- If `require_internet_for_actions = True`, the app MUST disable the action buttons and strictly block the driver from submitting operations offline to prevent timestamp/location manipulation.

11.2.4 Offline Sync Conflict Resolution (Field Execution Wins)

A race condition occurs if a Driver executes an action offline (e.g., delivers a shipment and collects COD), while a Tenant Admin simultaneously modifies or cancels that same shipment from the back-office web application before the driver reconnects to the internet.

- **Timestamp Tracking:** Every offline action queued in the Driver's SQLite database MUST capture the exact local device `execution_timestamp`.
- **Resolution Rule ("Field Execution Wins"):** When the background sync reaches the NestJS API, the backend MUST compare the Driver's `execution_timestamp` against the `updated_at` timestamp of the Shipment.
- If the shipment was modified/cancelled by an admin *after* the driver executed the action offline, the backend MUST NOT reject the driver's payload. Instead, it MUST accept the field data (especially if it contains COD financial ledgers or POD photos), and flag the Shipment record with `sync_conflict = True`.
- **UI Notification:** The Tenant Admin UI MUST display a prominent warning for conflicted shipments, requiring manual operational review, ensuring financial and physical realities on the ground are never discarded by the API.

11.3 Dynamic Operation Actions Engine

The React Native app MUST NOT hardcode action buttons like "Arrived" or "Loaded". Instead, it MUST render the workflow dynamically based on the `OperationActionConfig` fetched from the backend.

- If a flag (e.g., `is_photo_required`) is `True`, the app MUST block submission until the driver fulfills the requirement.

11.4 Hardware Access & Media Constraints

11.4.1 GPS & Geofencing

- If an action requires GPS, the app MUST capture high-accuracy coordinates.
- **Geofence Evaluation:** The app SHOULD calculate the distance between the device's current location and the planned `Destination_Point`. If the distance exceeds the configured `geofence_radius_meters`, the app MUST flag the `location_match_status` as `Mismatched`.
- **Manual Override:** If `allow_manual_location = True`, the app MUST provide a "Pin on Map" fallback.

11.4.2 Media Constraints

- **Gallery Restriction:** If `allow_gallery_upload = False`, the app MUST hide the "Upload from Gallery" option and force the driver to use the live camera.
- **Video Limitations:** Video recording MUST automatically stop when `max_video_duration_sec` is reached.

11.5 Cash on Delivery (COD) Strict Workflow

The COD workflow is a critical financial gate. The app MUST enforce the **"No Cash, No Delivery"** rule.

11.5.1 The Delivery Block

- If `order_type == Credit`, the app displays the "Complete Shipment" button.
- If `order_type == COD`, the app MUST hide the "Complete" button and display a green **"Collect Payment"** button.

11.5.2 Payment Collection Process

1. **Input:** The driver inputs the collected amount.
2. **Variance Check:** If the input amount is less than the `cod_amount`, the app MUST check the `allow_partial_cod` setting. If `False`, the app MUST block the submission.
3. **Ledger Update:** Upon successful submission, the amount MUST immediately reflect in the Driver's **"My Wallet"** tab.

   🛑 **CRITICAL RULE: COD Atomic Sync** When syncing a COD collection to the backend, the NestJS API MUST process it atomically: It MUST create the `ClientPayment` record, credit the Driver's Treasury Box, AND update the `Shipment.status` to `Delivered` in a **single database transaction**.

11.6 Visual & UX Mandates

- **Color Coding:** `Blue` = Navigation/Information, `Green` = Success/Collect Cash, `Orange` = Warning, `Red` = Error/Stop.
- **Offline Indicator:** A prominent **Yellow Bar** MUST remain pinned to the top of the screen whenever the app relies on the local SQLite database.

11.7 App Versioning & Force Update Mechanism

The backend API evolves over time. To prevent API contract breakages and data corruption caused by outdated mobile clients, the vendor MUST implement a strict version control mechanism.

- **Version Handshake:** Upon application launch (when an internet connection is available), the React Native app MUST query a public backend endpoint (e.g., `/api/v1/public/mobile-version`) to fetch the current `minimum_required_version` configured by the Super Admin.
- **The UI Lock:** If the app's current build version is lower than the `minimum_required_version`, the app MUST immediately lock the UI, display a mandatory "Update Required" screen, and provide a direct button linking to the respective App Store or Google Play Store.
- **Bypass Prohibition:** The driver MUST NOT be able to dismiss this update screen to access the offline database or continue operational workflows.

Part 12: Localization, Timezones & Formatting

12.1 Timezone Architecture (The UTC Standard)

Handling time correctly across a multi-tenant platform is critical. The vendor MUST implement the "UTC Everywhere" architectural pattern.

12.1.1 Database Storage (PostgreSQL)

- All date and time fields (`TIMESTAMP`) across the `master` and `tenant` schemas MUST be stored in Coordinated Universal Time (UTC) using the `TIMESTAMPTZ` data type.
- Storing localized times directly in the database is STRICTLY PROHIBITED.

12.1.2 Logical Conversion (NestJS & React)

- **API Layer:** The NestJS backend MUST send and receive all timestamps in standard ISO 8601 UTC format.
- **Presentation Layer:** The React and React Native frontends MUST convert the UTC timestamp to the user's local time for display purposes.

> 🛑 **CRITICAL RULE: Date-Only Fields** For purely commercial date fields (e.g., `issue_date` on an Invoice), the system MUST treat these as "Floating Dates". The backend MUST NOT shift an invoice issue date backwards to the previous day due to a UTC timezone conversion.

12.2 Internationalization (i18n) & RTL Support

- **Frontend:** The vendor MUST utilize a robust localization library (e.g., `react-i18next`). Hardcoding UI text strings is PROHIBITED.
- **RTL Enforcement:** When the system language is set to Arabic, the React frontend and React Native app MUST dynamically switch the layout direction to Right-to-Left (RTL).
- **Backend PDF Generation:** The NestJS PDF generation engine MUST natively support UTF-8 encoding and complex text rendering for Arabic script. Disconnected Arabic letters in PDFs constitute a severe bug.

12.3 Dynamic Formatting (Numbers & Dates)

The UI display must strictly adhere to the commercial preferences set in the `OrganizationProfile`.

| **Preference Key** | **Scope** | **Logic & Enforcement** |

| `date_format` | React/App UI | The frontend MUST format display dates exactly as selected (e.g., `YYYY-MM-DD` vs. `DD/MM/YYYY`). API expects ISO 8601. |

| `number_format` | React/App UI | Defines the thousands separator and decimal point (e.g., `1,234.56` vs. `1.234,56`). Database stores raw `DECIMAL`. |

| `negative_format` | Financial Reports | Defines how negative values are displayed (`-100` vs. `(100)`). |

12.4 Dual-Currency Display Rules

IRoad utilizes a Dual-Currency system. The UI/UX MUST enforce clarity to prevent financial errors.

- **Transaction Screens:** The UI MUST prominently display the *Transaction Currency* symbol next to the amounts.
- **Analytical Dashboards:** All aggregated KPI widgets (e.g., Total Revenue, MRR) MUST aggregate and display values exclusively in the *Base Currency*.
- **Conversion Visibility:** If a transaction is executed in a secondary currency, the UI MUST display the `fx_rate` used and the `base_amount` as read-only informational fields.

End of Volume III.

Volume IV: Development Standards & Quality Metrics

Part 13: API Strategy & Documentation

13.1 RESTful Architecture & Payload Standards

The vendor MUST expose all backend functionalities via a strict RESTful API architecture. GraphQL or GraphQL-like dynamic querying from the frontend is OUT OF SCOPE and MUST NOT be used in Phase 1.

13.1.1 HTTP Methods & Status Codes

The NestJS Controllers MUST adhere strictly to standard HTTP methods and return appropriate HTTP status codes.

HTTP Method	Operation Type	Required Success Code	Required Error Codes (Examples)
`GET`	Read / Fetch	`200 OK`	`404 Not Found`, `401 Unauthorized`
`POST`	Create / Execute Action	`201 Created`	`400 Bad Request` (Validation Failed), `409 Conflict`
`PUT`	Full Replace (Rarely Used)	`200 OK`	`400 Bad Request`
`PATCH`	Partial Update	`200 OK`	`404 Not Found`, `400 Bad Request`
`DELETE`	Soft/Hard Delete	`200 OK` or `204 No Content`	`403 Forbidden`, `409 Conflict` (If linked data exists)

13.1.2 Data Transfer Objects (DTOs) & Validation

The React and React Native frontends MUST NOT send unstructured JSON payloads.

- Every `POST`, `PUT`, and `PATCH` endpoint MUST possess a strongly typed TypeScript DTO class.
- The backend MUST utilize `class-validator` and `class-transformer` to sanitize and validate incoming payloads *before* they reach the service layer.
- **Stripping Unknowns:** The NestJS global `ValidationPipe` MUST be configured with `whitelist: true` and `forbidNonWhitelisted: true` to automatically reject payloads containing unexpected or malicious fields.

13.2 Tenant Context & Routing Security

IRoad is a multi-schema platform. The backend must unerringly identify which schema a request belongs to.

🛑 **CRITICAL RULE: Tenant Spoofing Prevention**

The React and React Native applications MUST NOT pass the `tenant_id` as a URL parameter, query string, or JSON body field for operational endpoints.

The NestJS backend MUST extract the `tenant_id` exclusively from the cryptographically verified JWT payload. Trusting a client-provided `tenant_id` is a critical security vulnerability and is STRICTLY PROHIBITED.

13.2.1 API Route Versioning & Namespacing

To allow future scalability and separate Admin flows from Tenant flows, the API routes MUST be prefixed logically:

- **Super Admin Routes:** `/api/v1/admin/...` (Requires Super Admin Role).
- **Tenant Operations Routes:** `/api/v1/tenant/...` (Requires Tenant User Role).
- **Driver Operations Routes:** `/api/v1/driver/...` (Requires Driver Role).
- **Public/Shared Routes:** `/api/v1/public/...` (e.g., Login, OTP request).

13.3 Pagination, Sorting & Filtering

High-density data tables (e.g., Bookings, Invoices) are central to the IRoad UX.

- All `GET` endpoints returning collections MUST support Pagination, Sorting, and Filtering via query parameters.
- **Pagination Standard:** The backend MUST implement Offset-based pagination (`?page=1&limit=50`).
- **Metadata Payload:** The JSON response for paginated endpoints MUST wrap the data in a standardized metadata object.

```
{
  "data": [
    { "shipment_no": "SH-1001", "status": "Delivered" }
  ],
  "meta": {
    "total_items": 150,
    "item_count": 50,
    "items_per_page": 50,
    "total_pages": 3,
    "current_page": 1
  }
}
```

13.4 API Documentation Mandate (Swagger/OpenAPI)

The vendor MUST NOT rely on external wikis or Word documents. The API documentation MUST live inside the backend source code and generate automatically.

- **Swagger Integration:** The NestJS backend MUST implement `@nestjs/swagger`.

- **Definition of Done:** An endpoint is NOT considered "Done" unless it is fully decorated with Swagger annotations.

```
// Example: Mandatory NestJS Controller Documentation Standard
@ApiTags('Tenant Shipments')
@ApiBearerAuth('JWT-auth')
@Controller('api/v1/tenant/shipments')
export class ShipmentsController {

  @Post()
  @ApiOperation({ summary: 'Create a new shipment' })
  @ApiResponse({ status: 201, description: 'Shipment created successfully.',
type: ShipmentResponseDto })
  @ApiResponse({ status: 400, description: 'Validation failed.' })
  async createShipment(@Body() dto: CreateShipmentDto, @Req() req) {
     // Business logic
  }
}
```

Part 14: Clean Code & Architecture Patterns

14.1 Strict TypeScript Enforcement

To eliminate runtime errors and ensure code maintainability, the vendor MUST utilize TypeScript across the entire stack in "Strict Mode".

- **Compiler Configuration:** The `tsconfig.json` MUST enforce `"strict": true` (including `noImplicitAny` and `strictNullChecks`).
- **Prohibited Practices:** Developers MUST NOT use the `any` type to bypass type checking. The use of `@ts-ignore` is STRICTLY PROHIBITED unless accompanied by a documented engineering justification.

14.2 Backend Architecture (NestJS Modular Pattern)

The backend MUST NOT mix HTTP routing, business logic, and database queries within the same file. The vendor MUST adhere strictly to the **Modular Service-Repository Pattern**.

14.2.1 Layer Responsibilities

1. **Controllers (``):** MUST only handle HTTP routing and DTO validation. Controllers MUST NOT contain business logic.
2. **Services (``):** MUST contain all business and workflow logic. Services MUST NOT execute direct SQL queries.
3. **Repositories (``):** MUST handle all data persistence and retrieval using the ORM.

```
graph TD
    A[HTTP Request] --> B[Controller: Validates DTO]
    B --> C[Service: Business Logic & Rules]
    C --> D[Repository: ORM/Query Builder]
    D --> E[(PostgreSQL)]
```

```
    E --> D
    D --> C
    C --> B
    B --> A[HTTP Response]
```

🛑 **CRITICAL RULE: Fat Models / Skinny Controllers**

If a Controller file exceeds 200 lines of code, or contains `if/else` statements executing business validations, it violates the architectural standard and SHALL be rejected during Code Review.

14.3 Frontend Architecture (React & React Native)

The React frontend and React Native Driver App MUST be built using **Component-Driven Design**. The UI layer MUST be decoupled from data-fetching and complex state mutations.

14.3.1 Custom Hooks & Data Fetching

- **Prohibited Practice:** The vendor MUST NOT write `fetch` or `axios` API calls directly inside `useEffect` blocks within visual React Components.
- **Mandatory Practice:** All API calls and asynchronous state management MUST be encapsulated within Custom Hooks (e.g., utilizing `React Query` or `RTK Query`).

14.4 Code Quality Gates (Linting & Formatting)

- **ESLint & Prettier:** The repository MUST contain standardized `.eslintrc` and `.prettierrc` configurations.
- **Pre-commit Hooks (Husky):** The vendor MUST configure `husky` and `lint-staged`. When a developer attempts to run `git commit`, the hook MUST automatically run ESLint and Prettier. If there are compilation errors or Linting violations, the commit MUST abort.

Part 15: Testing Strategy & DB Migrations

15.1 Automated Testing Mandate

Manual Quality Assurance (QA) alone is insufficient. The vendor MUST deliver a comprehensive suite of automated tests.

15.1.1 Unit Testing (Business Logic)

The vendor MUST utilize **Jest** to write isolated unit tests for the NestJS `Services` layer.

The following core business logic domains MUST achieve a minimum of **85% test coverage**:

1. Financial Calculations (Taxes, Dual-Currency FX Rates).
2. Pro-Rata Billing Math.
3. Settlement Logic (FIFO application of ClientPayments).
4. State Machine Transitions (Ticket Ping-Pong, Shipment status guards).

15.1.2 End-to-End (E2E) Testing

The vendor MUST write E2E tests using `Supertest` to validate critical user journeys:

- **Tenant Provisioning Flow:** Confirming schema creation, DB migrations, and seeding.
- **COD Collection Flow:** Verifying the atomic transaction of collecting COD and updating the driver wallet.
- **Kill Switch Test:** Verifying that a suspended tenant immediately receives `401 Unauthorized` on active tokens via Redis eviction.

> 🛑 **CRITICAL RULE: CI Pipeline Blocking**
>
> The automated test suite MUST be integrated into the CI/CD pipeline. If any test fails, or if the coverage drops below the mandated threshold, the pipeline MUST abort the build.

15.2 Multi-Schema Database Migrations

Standard ORM migration tools only run against a single target schema by default.

15.2.1 The Migration Runner Architecture

The vendor MUST engineer a custom Database Migration Runner script within the NestJS application.

- **Master Migrations:** MUST execute first, applying changes only to the `master` schema.
- **Tenant Migrations:** The runner MUST query the `master.Tenants` table to retrieve all active tenant schemas, loop through each sequentially, and apply the tenant-specific SQL migrations.

```
sequenceDiagram
    autonumber
    participant CI as CI/CD Pipeline
    participant Runner as NestJS Migration Runner
    participant DB_Master as Postgres (Master)
    participant DB_Tenants as Postgres (Tenants)

    CI->>Runner: 1. Execute `npm run migration:up`
    Runner->>DB_Master: 2. Apply Master Migrations
    Runner->>DB_Master: 3. Query active_tenants
    DB_Master-->>Runner: 4. Returns ['tenant_a1', 'tenant_b2']
    loop For Each Tenant Schema
        Runner->>DB_Tenants: 5. SET search_path TO `tenant_X`
        Runner->>DB_Tenants: 6. Apply Tenant Migrations
    end
    Runner-->>CI: 7. Process Complete (Exit Code 0)
```

15.2.2 Zero Data-Loss Mandate

Migrations MUST NOT utilize destructive operations (`DROP TABLE`, `DROP COLUMN`) during Phase 1 without explicit written approval.

15.2.3 Migration Failure Resiliency & Tracking

When executing migrations across 100+ isolated tenant schemas, a single tenant's corrupted data might cause their specific migration to fail.

- **Fault Tolerance:** The Migration Runner MUST NOT crash and halt the entire deployment process if one tenant schema fails.
- **State Tracking:** The runner MUST log the migration status per tenant in a `master.SchemaMigrationsLog` table.
- If `tenant_41` fails, the runner MUST log the error, skip it, and proceed to successfully migrate `tenant_42` through `tenant_100`.

Part 16: Observability, APM & Global Error Handling

16.1 Global Error Handling Architecture (Backend)

16.1.1 The Global Exception Filter

- The NestJS application MUST utilize a global `ExceptionFilter` (`@Catch()`).
- **Prohibited Practice:** Developers MUST NOT write `try/catch` blocks inside controllers that return raw `res.status(500).send(error.message)`.

> 🛑 **CRITICAL RULE: Stack Trace Leakage Prohibition**
>
> The backend MUST NOT expose internal application stack traces, PostgreSQL query syntax, or database schema details in the HTTP response under any circumstances. All internal database errors MUST be caught, logged internally, and transformed into a generic message for the client.

16.1.2 Standardized Error Response Payload

The backend MUST return all HTTP errors in a strictly unified JSON structure.

```
{
  "statusCode": 400,
  "timestamp": "2026-03-15T14:30:00.000Z",
  "path": "/api/v1/tenant/shipments",
  "errorCode": "ERR_VALIDATION_FAILED",
  "message": ["unit_price must be a positive number"],
  "traceId": "req-a1b2c3d4"
}
```

16.1.3 Business Logic Exceptions & UX

The vendor MUST strictly differentiate between Technical Errors (e.g., DB connection loss) and Business Rule Violations (e.g., "Credit Limit Exceeded", "Invalid Geofence").

- Business rule violations MUST NOT return a `500 Internal Server Error`. They MUST return an appropriate HTTP status code such as `422 Unprocessable Entity`, `409 Conflict`, or `400 Bad Request`.

- The `errorCode` in the JSON payload MUST be specific, standardized, and actionable (e.g., `ERR_CREDIT_LIMIT_EXCEEDED`).
- The Frontend (React/React Native) MUST intercept these specific error codes and translate them into "Operational Warnings" (e.g., a localized, user-friendly Toast or Modal clearly explaining the business constraint) rather than displaying them as technical system crashes.

16.2 Centralized Structured Logging

- The vendor MUST implement asynchronous, structured JSON logging (e.g., `Winston` or `Pino`).
- Every log entry (Warning, Error, Info) MUST automatically append the Request Context: `tenant_id`, `user_id`, and a unique `traceId`.

Log Field	Data Type	Description
`level`	`Enum`	`[info, warn, error, debug]`
`message`	`Text`	The human-readable log description.
`traceId`	`Text`	A unique UUID generated per HTTP request to trace the flow across services.
`tenantId`	`Text`	Extracted from the JWT to isolate tenant-specific errors.
`stack`	`Text`	The full technical stack trace (Only appended if `level = error`).

16.3 Application Performance Monitoring (APM)

- The vendor MUST integrate an APM tool (e.g., **Sentry**, **Datadog**) into both the NestJS backend and the React frontends.
- Any `UnhandledPromiseRejection` MUST be captured by the APM tool, trigger an alert, and gracefully shut down the worker thread to prevent memory leaks.

16.4 Frontend Resilience (React & React Native)

- **React Error Boundaries:** The React architectures MUST implement Error Boundaries. If a component crashes, it MUST render a standardized "Fallback UI" while keeping the Side Navigation functional.
- **API Failure Handling:** The frontend HTTP client MUST implement a global response interceptor. If the backend returns `401 Unauthorized`, the interceptor MUST instantly clear local storage and redirect to Login.

Part 17: Non-Functional Requirements (NFRs) & Performance

17.1 Availability & Uptime (SLA Baseline)

- **Target Uptime:** The system architecture MUST support a **99.9% uptime** SLA.
- **Stateless API Design:** The NestJS backend MUST be strictly stateless. No session data may be stored in Node.js memory. All state MUST be delegated to PostgreSQL and Redis.

17.2 Performance & Latency Metrics

The backend MUST be load-tested to guarantee the following response times (measured at API Gateway level):

Operation Type	Target P95 Latency	Example Endpoints
Authentication & AuthZ	`< 100 ms`	Login, Token Refresh, Redis Session Check
Standard CRUD (Read/ Write)	`< 300 ms`	Create Shipment, Update Booking, List Drivers
Heavy Aggregation / Reports	`< 800 ms`	Generate Job Order Report, Financial Ledger

> 🛑 **CRITICAL RULE: Database Query Optimization (N+1 Problem)**
>
> The NestJS backend MUST NOT suffer from the "N+1 Query Problem". Fetching a list of 50 Shipments with their associated Client Names MUST execute in a single, optimized SQL `JOIN` or batched query.

17.3 Scalability & Concurrency

- **Connection Pooling:** The NestJS application MUST utilize a robust database connection pooler (e.g., PgBouncer or internal ORM pooling).
- **Queue-Based Scaling:** Any operation that takes longer than 1.5 seconds MUST be offloaded to a Redis-backed background queue (BullMQ). The HTTP request MUST return a `202 Accepted` immediately.

17.4 Driver App Mobile Constraints (React Native)

Drivers use low-end Android devices with limited data plans.

Constraint	Limit / Rule	Technical Mitigation
Battery Consumption	Minimal Drain	Background GPS tracking MUST NOT poll continuously unless explicitly required for a Geofence check. Background syncs MUST utilize OS-level batching.
Data Usage	Highly Optimized	Image uploads (Digital POD) MUST be compressed (e.g., max 1080p, `< 1MB`) *before* uploading to S3 over cellular networks.
Local Storage	`< 50 MB`	The local SQLite database MUST auto-prune "Completed" jobs and synced logs older than 7 days.

End of Volume IV.

Volume V: Deployment & Handover Protocol

Part 18: CI/CD Pipelines & Environments

18.1 Environment Strategy

The vendor MUST design the infrastructure and deployment scripts to support three strictly isolated environments. Manual code modifications (e.g., FTP uploads or direct SSH file editing) to the Staging or Production environments are STRICTLY PROHIBITED.

Environment	Purpose & Infrastructure	Data Source	Target Audience
Development (Dev)	Active branch integration. Auto-deploys on `develop` branch merges.	Seeded Mock Data	Development Team
Staging (UAT)	Exact replica of Production architecture (Dockerized). Used for final sign-off.	Sanitized Sandbox Data	Platform Owner & QA
Production (Prod)	Live operational SaaS environment. Requires manual trigger/approval to deploy.	Live PostgreSQL DB	Transporters & Super Admins

18.2 Continuous Integration (CI) Quality Gates

The repository MUST contain configuration files for an automated CI/CD pipeline (e.g., `.github/workflows/deploy.yml` or `.gitlab-ci.yml`).

Whenever a developer opens a Pull Request (PR) against the `main` or `develop` branches, the CI pipeline MUST execute the following Quality Gates sequentially:

1. **Linting & Formatting Check:** Run `eslint` and `prettier` checks.
2. **Type Checking:** Execute `tsc --noEmit` to catch strict TypeScript compilation errors.
3. **Automated Testing:** Execute the Jest Unit Tests and Supertest E2E Tests (as mandated in Part 15).
4. **Security Scanning:** Run `npm audit` to detect known vulnerabilities in open-source dependencies.

   🛑 **CRITICAL RULE: Pipeline Blocking (The Zero-Tolerance Policy)**

   If ANY of the four Quality Gates fail, or if the Unit Test coverage drops below 85%, the CI pipeline MUST immediately `FAIL` and block the Pull Request from being merged. The vendor MUST NOT bypass the CI pipeline using admin privileges.

18.3 Continuous Deployment (CD) Workflow

Once the code is merged into the target branch, the CD pipeline automates the delivery process. The vendor MUST containerize the applications and push them to a secure Container Registry.

```
sequenceDiagram
    autonumber
    participant Git as Git Repository
    participant CI as CI Runner
    participant Reg as Docker Registry
    participant Server as Target Server (Staging/Prod)

    Git->>CI: 1. Merge to `main` branch
    CI->>CI: 2. Execute Quality Gates (Tests pass)
    CI->>CI: 3. Build Docker Images (Backend & Frontend)
    CI->>Reg: 4. Push tagged images (e.g., `iroad-api:v1.2.0`)
    CI->>Server: 5. SSH / Trigger Webhook to Server
    Server->>Reg: 6. Pull latest images
    Server->>Server: 7. Execute `npm run migration:up` (DB Sync)
    Server->>Server: 8. Execute `docker-compose up -d`
    Server-->>CI: 9. Deployment Successful
```

18.4 Database Migration Automation in CI/CD

Applying database schema changes in a multi-tenant environment during deployment carries a high risk of downtime or data corruption.

- **Execution Order:** The CI/CD script MUST execute the custom NestJS Migration Runner (defined in Part 15) *before* restarting the main application containers.
- **Master First, Tenants Second:** The script MUST successfully migrate the `master` schema before attempting to migrate the `tenant_{id}` schemas.
- **Failure Handling:** If the migration script fails on any tenant, the deployment pipeline MUST abort, alert the engineering team, and NOT restart the Node.js containers, leaving the previous stable version running.

18.4.1 API Backward Compatibility Mandate

Because of the Fault-Tolerant Migration Runner, a deployment might result in a state where some tenants are on the "New Schema Version" and others are stuck on the "Old Schema Version" pending manual DBA intervention.

- **Code Resilience:** The NestJS API deployment MUST be strictly **Backward Compatible**. The backend code MUST gracefully handle interactions with both the old database structure and the new database structure simultaneously until all tenant schemas are fully synchronized.

18.5 Secrets & Configuration Management

Security credentials MUST NOT be tracked in Version Control.

- **Pipeline Secrets:** Sensitive variables (e.g., DB Passwords, JWT Secrets, Payment Gateway Master Keys) MUST be stored securely within the CI/CD provider's secret manager (e.g., GitHub Secrets, AWS Systems Manager Parameter Store).
- **Environment Injection:** The CD pipeline MUST dynamically inject these secrets into the `.env` files or directly into the Docker containers during the deployment phase.

📋 **Testability & Definition of Done:**

Upon Code Handover, the Platform Owner will inspect the Git repository. The deliverable is accepted ONLY IF:

1. The repository contains functioning CI/CD YAML files.
2. The Platform Owner can observe past PRs where the CI pipeline successfully blocked a merge due to a failing Unit Test.
3. The `docker-compose.prod.yml` file is completely devoid of hardcoded passwords.

Part 19: Handover, Source Code IP & Definition of Done (DoD)

19.1 Intellectual Property (IP) & Source Code Ownership

The IRoad platform is a proprietary commercial system. The vendor MUST ensure that all deliverables seamlessly transfer full technical and legal ownership to the Platform Owner.

- **Repository Ownership:** All source code MUST be committed directly to a Git repository (e.g., GitHub, GitLab) owned and controlled by the Platform Owner. The vendor MUST NOT host the primary repository on their own infrastructure.
- **No Obfuscation:** The delivered source code MUST be completely readable and unminified. The vendor MUST NOT use code obfuscators, binary compilers (for the Node.js source), or encrypted logic blocks.
- **No Vendor Lock-in:** The vendor MUST NOT utilize proprietary, closed-source libraries owned by the vendor themselves without explicit written consent. All third-party dependencies MUST be publicly available via standard package managers (e.g., `npm`) under permissive licenses (e.g., MIT, Apache 2.0).

🛑 **CRITICAL RULE: Backdoor & Telemetry Prohibition**

The delivered source code MUST NOT contain any hidden "backdoors", remote kill switches controlled by the vendor, or unauthorized telemetry/analytics tracking that sends data to the vendor's servers.

19.2 Definition of Done (DoD)

To prevent disputes over whether a module or feature is "finished", the vendor MUST adhere to this strict Definition of Done. The Platform Owner will reject any Pull Request (PR) or Module Handover that fails to check ALL of the following boxes.

The Mandatory DoD Checklist:

- [ ] **Code Complete:** All functional requirements defined in the TRD and PCS documents are implemented.
- [ ] **CI/CD Quality Gates Passed:** The code passes ESLint, Prettier, and strict TypeScript compilation (`tsc --noEmit`) without bypassing rules.
- [ ] **Automated Tests Passed:** Jest Unit tests and Supertest E2E tests execute successfully.
- [ ] **Coverage Threshold Met:** The business logic layers (NestJS Services) achieve at least **85%** test coverage.

- [ ] **Database Migrations Created:** Any changes to the database schema are codified in migration scripts (e.g., Prisma/TypeORM) and tested using the Multi-Schema Migration Runner.
- [ ] **Swagger Documented:** All new REST API endpoints are decorated with `@nestjs/swagger` and visible on the `/api/docs` route, containing correct DTOs and authorization headers.
- [ ] **No N+1 Queries:** Database queries are optimized, utilizing proper `JOIN`s or batched fetching.
- [ ] **Docker Ready:** The application builds and runs successfully via `docker-compose up` without manual configuration steps.

19.3 Handover Protocol & Documentation

A working system is useless if the Platform Owner's internal engineering team cannot operate it. The vendor MUST deliver comprehensive technical documentation alongside the source code.

19.3.1 The Master `README.md`

The root of the repository MUST contain a detailed `README.md` file instructing developers on how to bootstrap the environment from scratch. It MUST include:

1. **Prerequisites:** e.g., Docker, Node.js version, npm version.
2. **Environment Setup:** Clear instructions on copying the provided `.env.example` file and the meaning of critical variables.
3. **Bootstrapping Commands:** The exact CLI commands to start the databases, apply migrations, seed the `master` schema, and start the frontend/backend servers locally.
4. **Testing Commands:** How to execute the Unit and E2E test suites.

19.3.2 The Infrastructure Runbook

The vendor MUST provide a "Runbook" documenting the deployment architecture.

- How to execute the Multi-Schema database migrations in a production environment.
- How to securely inject API keys (Payment Gateways, SMS) into the CI/CD pipeline secrets manager.
- How to backup and restore a specific `tenant_id` PostgreSQL schema.

19.4 Final Acceptance Testing (UAT) & Sign-Off

The Handover is not an event, but a verifiable process.

```
sequenceDiagram
    autonumber
    participant Vendor as Development Vendor
    participant Git as Platform Owner's Git
    participant CI as CI/CD Pipeline
    participant PO as Platform Owner

    Vendor->>Git: 1. Push Final Release Candidate (RC)
    Git->>CI: 2. Trigger Automated Quality Gates
    CI-->>Vendor: 3. Quality Gates Pass (Tests, Linting)
    Vendor->>PO: 4. Submit Formal Handover Request
    PO->>Git: 5. Conduct Code Review & Architecture Audit
```

```
PO->>CI: 6. Deploy to Staging Environment via Docker
PO->>PO: 7. Execute UAT (Test Kill Switch, COD, Isolation)
PO-->>Vendor: 8. Official Sign-off & Contract Fulfillment
```

📋 **Testability & Final Acceptance:**

During Step 7 (UAT), the Platform Owner will rigorously test the system's defensive architecture as defined in this TRD.

- If a Tenant can query another Tenant's data, the handover is rejected.
- If the Redis "Kill Switch" fails to immediately log out a suspended tenant, the handover is rejected.
- If the COD logic permits a delivery without a corresponding financial ledger entry, the handover is rejected.

Only when all TRD constraints are met will the final sign-off be granted.

End of Volume V. End of the IRoad Technical Requirements Document (TRD).